
Seguimiento

Trabajo Práctico Final

Algoritmos y Estructuras de Datos

- Antes de empezar a trabajar en el código, ya nos habíamos decidido a empezar a implementar el método **BuscarConOrden**, ya que teníamos mayor experiencia con estos tipos de algoritmos y, a partir de la finalización de este, podríamos saber que tener en cuenta para los demás.

Con respecto a los algoritmos usados, optamos por comenzar con una **Búsqueda Secuencial** y luego con un **Bubble Sort**, no porque sean los óptimos, sino porque ambas tienen una complejidad $O(N^2)$, lo que haría que se note aún más la diferencia al momento de realizar la “consulta 1”, además de ser más simples de implementar.

- Al momento de implementar **BuscarConOrden**, el único problema que encontramos al momento de llenar la lista **Collected**, ya que el conteo de cada elemento era correcto, pero se agregaba varias veces a la lista.

-Ejemplo:

List<string>:

rojo

azul

rojo

----->

List<dato>:

rojo, 2 ocurrencias

azul, 1 ocurrencia

rojo, 1 ocurrencia

rojo termina apareciendo 2 veces, creando un dato innecesario.

- Para resolverlo, se creó “**bool existe = false**” al principio, para revisar si el dato ya existe dentro de la lista, realizando un **break** si ya existe y un **continue** si no existe, agregándolo a la lista.
- Luego, nos dimos cuenta de que no usamos el parámetro “**cantidad**” (cantidad de resultados que se quieren ver). Nuestra solución fue usar “**remove range**” y borrar todo lo que este delante de los resultados que queremos ver.
- Una manera de mejora esta parte del código sería que cargue los datos más repetidos al **collected**, pero para eso habría que reestructurar todo el código...

- Tuvimos problemas al hacer el **Heap Sort**, ya que los únicos que hicimos recibían arreglos de enteros. No estamos acostumbrados a usar genéricos, pero es la mejor manera de implementarlo.
- Se agrego una variable **bool maxHeap** dentro de la clase **heap** donde se puede elegir si se ordena de forma ascendente o descendente (**maxHeap** o **minHeap**).
- La idea es optimizar la cantidad de operaciones para que se note la diferencia en la complejidad al hacer la **consulta 1**. la solución que encontramos fue usar la estructura **Dictionary**:

```
dictionary<string, int> contador = new dictionary <string, int>
```

Complejidad $O(N)$, a diferencia de un $O(N^2)$ de la otra busqueda.

- Creamos una clase **Heap** y se la puso en estructura.

- Hubo un problema al implementar **dictionary**, ya que la clase **heap** usa arreglos, mientras que los métodos en la clase **estrategia** utilizan listas. Eso se puede solucionar convirtiendo el **collected** y los datos a un **array**, para luego, convertir de nuevo la lista.
- Al armar el **heap** se usaron genéricos comparables, pero no hicimos que la clase **dato** sea comparable, así que se le agrego:

```
public class dato: Icomparable<dato> {
    .
    .
    .
    Public int compareTo(dato otro);
    return ocurrencia.CompareTo(otro.ocurrencia); }
```

- Para la **consulta 1** descubrimos que existe la estructura **StopWatch**, que sirve como un cronometro. La idea sería:

1) iniciar cronometro.	<code>StopWatch sw = new StopWatch();</code>
2) ejecutar método.	<code>StopWatch.start;</code>
3) parar cronometro.	<code>buscar con heap(...);</code>
4) guardar tiempo.	<code>tiempo = sw.elapsedmilliseconds;</code>

BuscarConHeap debería ser más rápido ya que su complejidad está compuesta por:

Dictionary $O(N)$
 +
HeapSort $O(n \log n)$

Mientras que **BuscarConOtro** está compuesto por:

BubbleSort

+

Doble for = $O(n^2)$

Aunque, esta diferencia de tiempo debería ser más notoria mientras más grande sea el archivo.

En nuestro caso, usando un archivo pequeño dentro del mismo proyecto, siempre veíamos mínimo una diferencia de 1ms.

- En la consulta 2, al inicio nos mareamos un poco. Ya que, si bien anteriormente habíamos resuelto consignas similares en las practicas, lo hicimos con Arboles Binarios (Simplemente ir de manera recursiva al hijo izquierdo). Pero luego entendimos que lo que busca la consigna es que trabajemos con las propiedades de índice que tiene el Heap:

Para cada Índice en el Arreglo del Heap

Hijo Izquierdo = $2 * \text{índice} + 1$

Hijo Derecho = $2 * \text{índice} + 2$

Padre = $(\text{índice} - 1) / 2$

Una vez entendimos esto, solo hacía falta implementar los datos de entrada del “BuscarConHeap()”, los cuales son *Dictionary<string, int> conteo* y *List<Dato> collected*. Usando el conteo para llenar los collected de objetos tipo “Dato”.

Con el collected lleno, fuimos repitiendo la propiedad de Heap para encontrar al Hijo izquierdo hasta llegar a una hoja, y durante la recursión cada vez que se actualizaba el Hijo izquierdo se agregaba el dato al string “resultado”.

- Para la consulta 3 encontramos que era más fácil abordarla de la misma manera que la consulta anterior. Usamos BuildHeap(), ya que si usáramos HeapSort() nos devolvería un arreglo ordenado, y los mismos datos de entrada mencionados en la consulta previa.

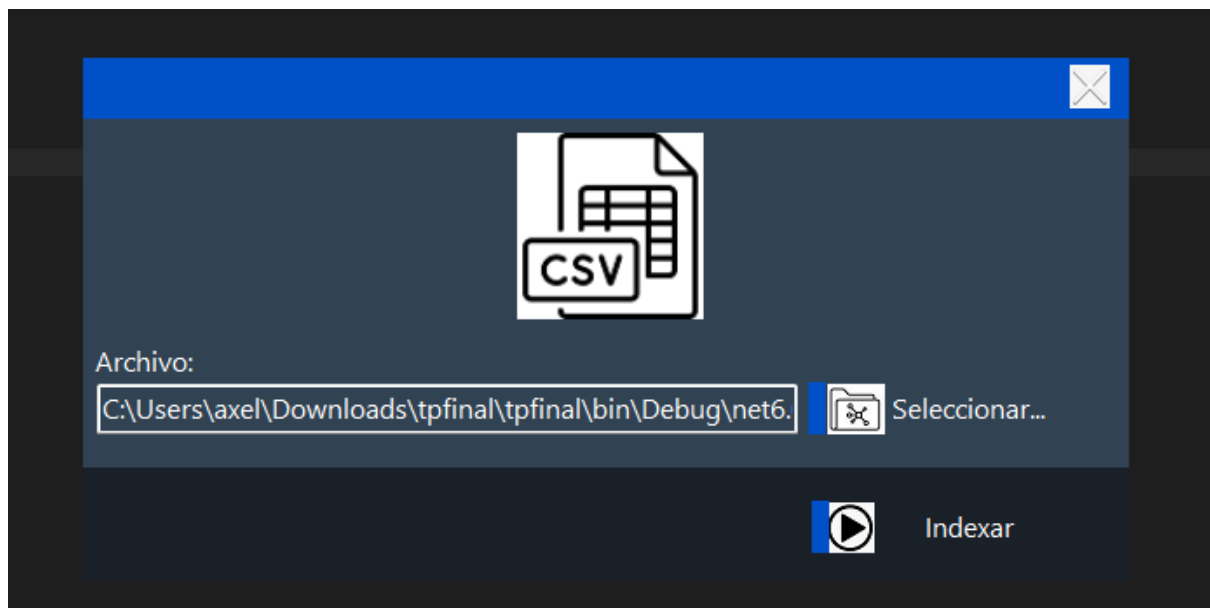
El único inconveniente que encontramos fue cuando teníamos que imprimir los niveles, eventualmente nos encontramos más cómodos con usar una variable “Nivel”, esta misma esta igualada a la fórmula de Heap para conseguir el Nivel usando el índice del arreglo (Similar a como antes usamos las propiedades de índice para los Hijos izquierdos).

$$Nivel = [\log_2 (\text{índice} + 1)]$$

Para esto usamos la clase “Math”, la cual nos permitió calcular de manera Logarítmica haciendo `(int)Math.Floor(Math.Log(i + 1, 2))`

Una vez aplicado el código y resueltos los problemas antes mencionados, podemos ejecutar el programa.

Al ejecutar el programa tenemos la primera pantalla que compone el sistema codificado, la pantalla de selección de archivo:



al seleccionar un archivo csv, pasamos a la pantalla principal en la que podremos seleccionar entre los métodos BuscarConHeap y BuscarConOtro para ordenar los datos que más se repiten dentro del archivo de menor a mayor, seleccionando la cantidad de resultados con una barra que va del 1 al 5

Metodo

☒ Heap ☐ Otro

Resultados: 1

Buscar

Limpiar

Consultas

Consulta 1

Consulta 2

Consulta 3

Metodo

☒ Heap ☐ Otro

Resultados: 5

Buscar

Limpiar

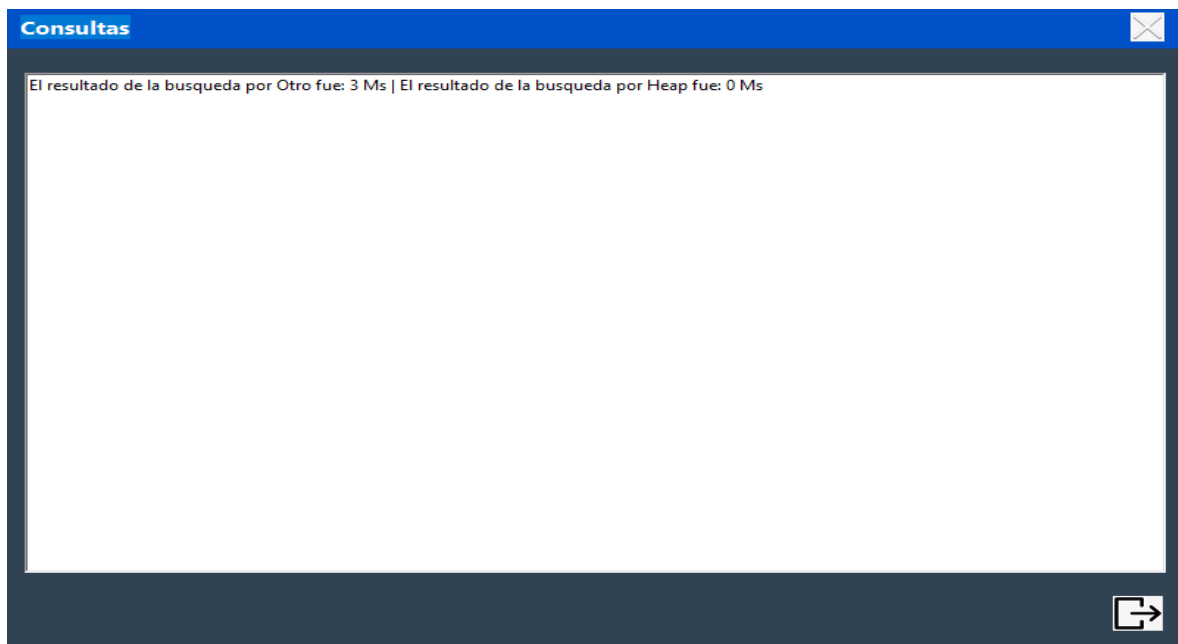
Consultas

Consulta 1

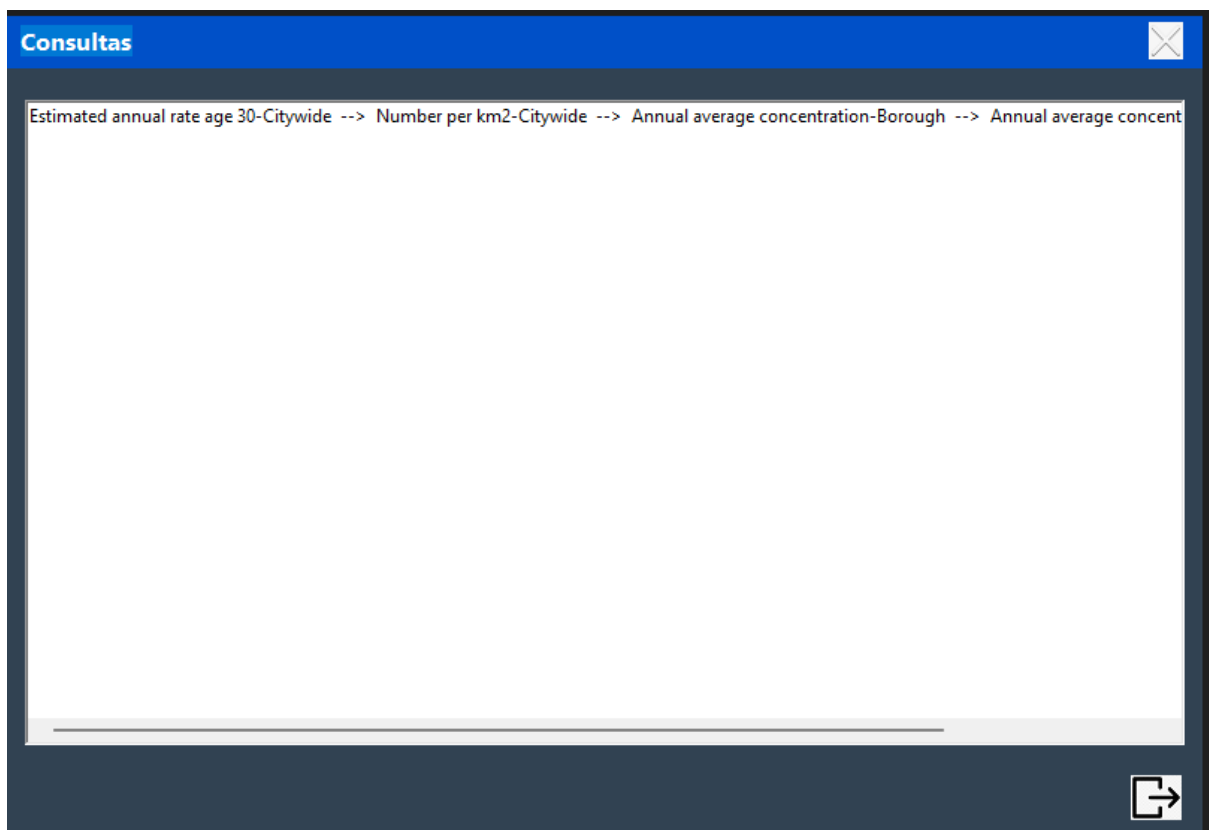
Consulta 2

Consulta 3

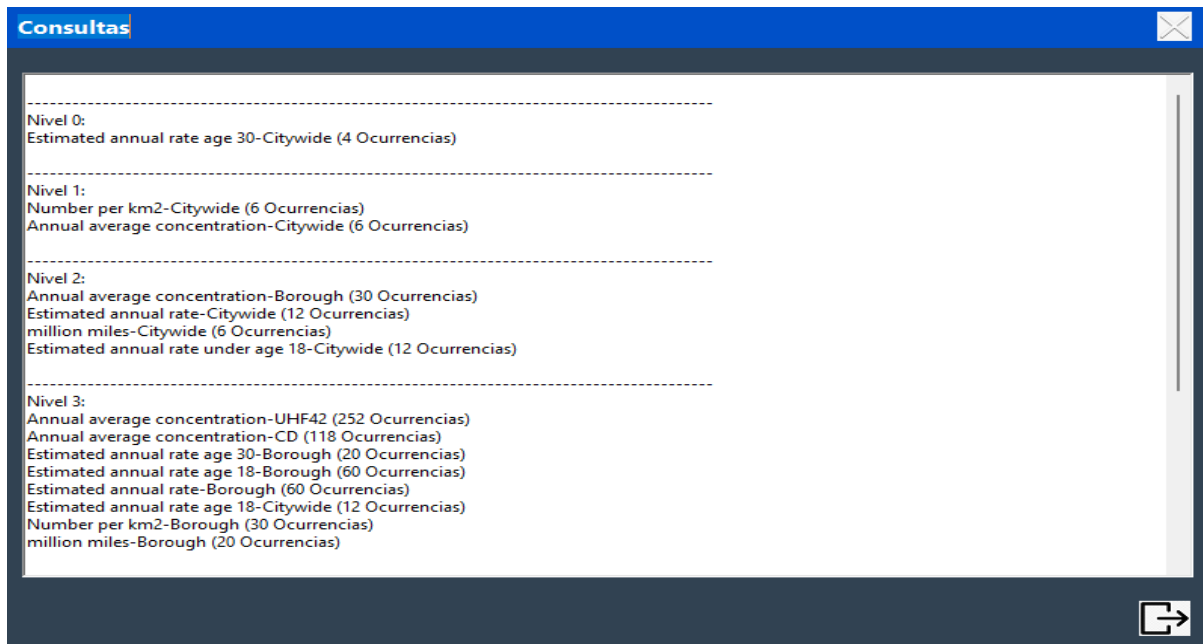
Cuando se selecciona la Consulta 1, se abre el menú de consultas, mostrando el tiempo en milisegundos que tarda `BuscarConHeap` y `BuscarConOtro` en realizar la búsqueda de los 5 elementos con mayor cantidad de ocurrencias:



Cuando se ejecuta Consulta 2 se abre el menú de consultas, mostrando el nombre de todos los datos que están en el camino hacia la hoja más izquierda:



Finalmente, cuando se ejecuta la Consulta 3, se muestra en el menú de consultas todos los datos dentro del archivo en sus respectivos niveles (con nuestro archivo se vieron 4 niveles):



Mejoras a futuro

- Fijándose más que nada en lo estético más que lo funcional, se podría mejorar la apariencia de los menús. Mas que nada las ventanas de consulta son muy planas y se asimilan mucho a un block de notas (Hasta incluso se puede escribir adentro, lo cual no debería porque poderse).
- Se podría expandir más en la cantidad de resultados visibles al buscar con algún método, ya sea ver más de cinco resultados o que directamente no haya un límite y se puedan ver todos los datos con sus respectivas ocurrencias.
- Ya que la clase Heap cuenta con una variable booleana “MaxHeap”, se podría implementar la opción de mostrar los resultados de manera ascendiente o descendiente dependiendo de cómo quiera el usuario (Ver los 5 con más ocurrencias y ver los 5 con menos ocurrencias).

Reflexión de la experiencia adquirida

Mientras implementábamos los métodos y las consultas pertenecientes a este trabajo pudimos ganar mucha experiencia para otros aspectos de la cursada, además de que nos sirvió para ir practicando lo aprendido para el parcial práctico.

Al mismo tiempo nos permitió ver la diferencia que hay, tanto en cantidad de operaciones como en tiempo de ejecución, entre los distintos algoritmos de búsqueda y ordenamiento.

A partir de este trabajo, también se nos hizo más fácil visualizar la infinidad de casos en los que podríamos aplicar los mismos métodos usados en el proyecto. Uno de los primeros ejemplos que se nos vino a la cabeza es el de un Navegador de Servidores de un videojuego, con cualquiera de los algoritmos podríamos ordenar los servidores de manera descendiente en base a la cantidad de personas conectadas, a la latencia que tengamos en el servidor, el nombre que tenga, etc.

En conclusión, la experiencia brindada al completar los métodos del trabajo práctico nos sirvió para poder cerrar y entender todos los algoritmos de recorrido, búsqueda y ordenamiento. Como también pudimos aprender distintos usos de otras estructuras e interfaces como Dictionary, Math, T, IComparable, Pila y Cola, más que nada haciendo énfasis en los primeros 4 mencionados.